Pattern Ricorsione: Divide et Impera

Quando usarlo: Il problema principale può essere tagliato a metà (o in *N* parti più piccole). Ogni parte viene risolta individualmente e, alla fine, i risultati parziali vengono fusi insieme per formare la soluzione globale.

Lezioni di riferimento: Lunedì (Quicksort, Ricerca Dicotomica).

Caratteristica chiave: L'algoritmo non fa uso di variabili globali o di stato persistente. Il risultato viaggia dal basso verso l'alto attraverso la catena dei `return`.

Template Python

```python
def divide_et_impera(problema):

    # 🚨 1. CASO BASE (Condizione di Terminazione)
    # Il problema è diventato così piccolo da essere banale?
    # Es. La lista da ordinare ha 0 o 1 elementi.
    if problema_is_trivial(problema):
        return risolvi_caso_banale(problema)

    else:
        # ⚠️ 2. DIVIDI (Scomposizione)
        # Spezza il problema in frammenti indipendenti.
        # Es. Scelgo un pivot e divido in "minori" e "maggiori".
        sub_problems = dividi_problema(problema)
        sub_solutions = []

        # 🚨 3. RISOLVI RICORSIVAMENTE
        for sub in sub_problems:
            # La funzione chiama se stessa sui singoli frammenti
            # Il risultato della discesa viene salvato!
            risultato_parziale = divide_et_impera(sub)
            sub_solutions.append(risultato_parziale)

        # ⚠️ 4. COMBINA E IMPERA (Riassembla i risultati)
        # Unisci i risultati parziali nella soluzione finale.
        soluzione_finale = combina_risultati(sub_solutions)

        return soluzione_finale
```

Note d'Esame

- **Non dimenticare il Return:** Sia il caso base che la fase di combinazione **devono** restituire un valore. Se ometti un `return`, otterrai un fatale `NoneType error` ai piani superiori dell'albero.
- **La Combinazione:** In algoritmi come il Quicksort, il passo 4 (Combinazione) si traduce in una banale concatenazione di liste con l'operatore `+` (es. `return minori + uguali + maggiori`).
- **Ottimizzazione:** L'efficienza dipende dal passo 2 (Dividi). Se il problema viene diviso esattamente a metà, l'algoritmo raggiunge la massima velocità.